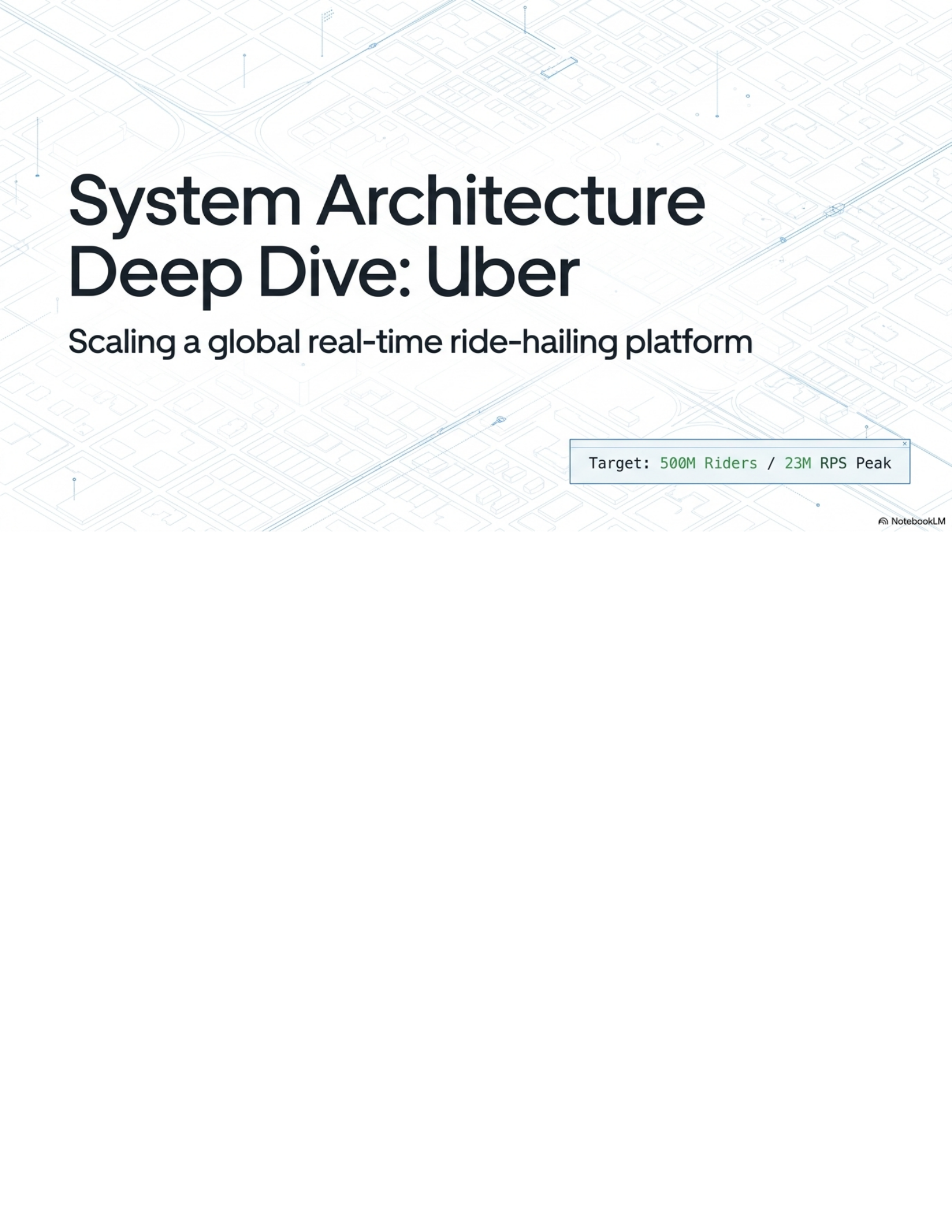


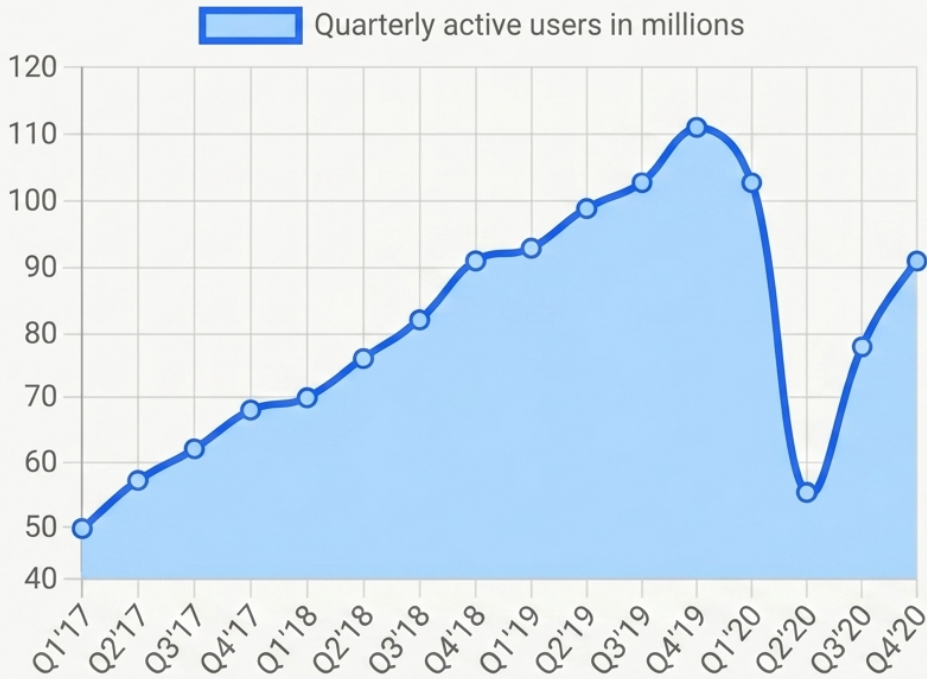


System Architecture Deep Dive: Uber

Scaling a global real-time ride-hailing platform

Target: 500M Riders / 23M RPS Peak

The Scale of the Problem



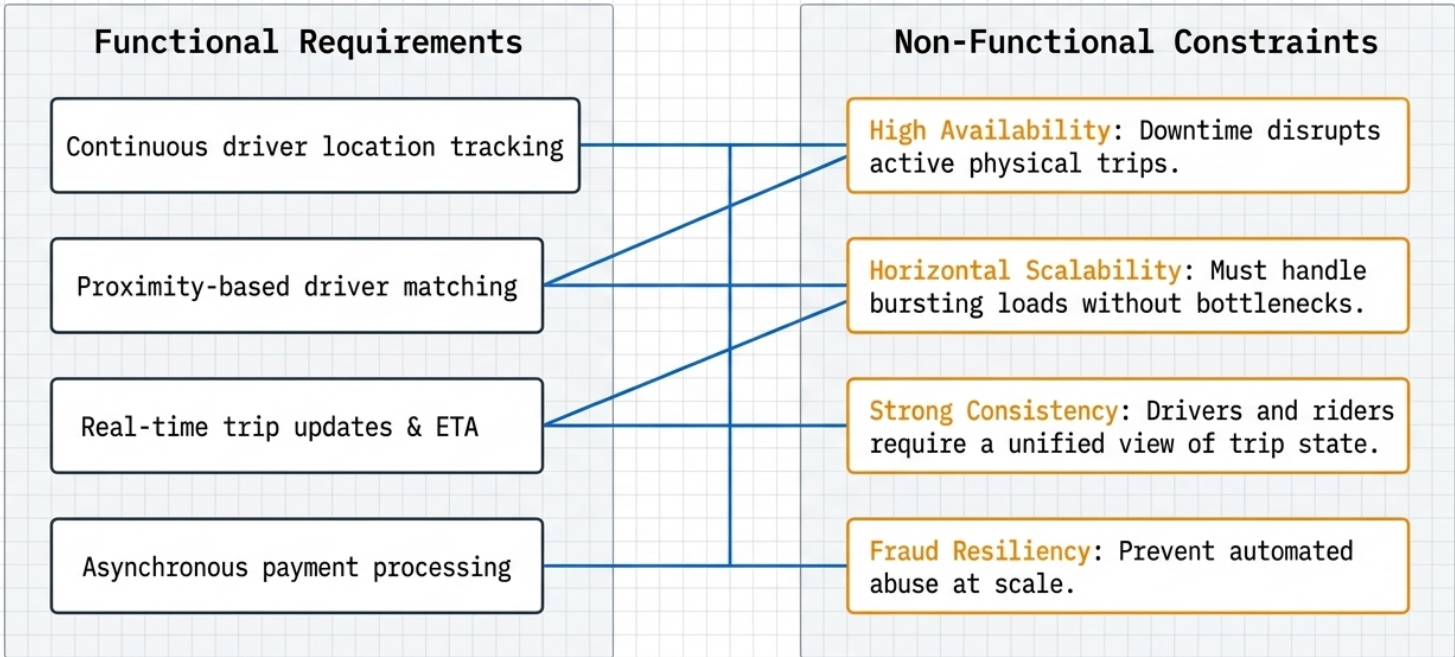
Total Riders
500 Million (20M Daily Active)

Total Drivers
5 Million (3M Daily Active)

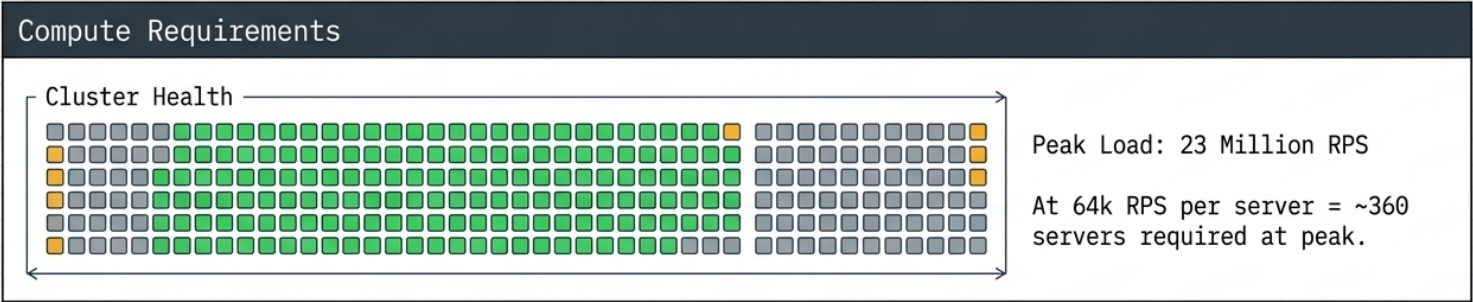
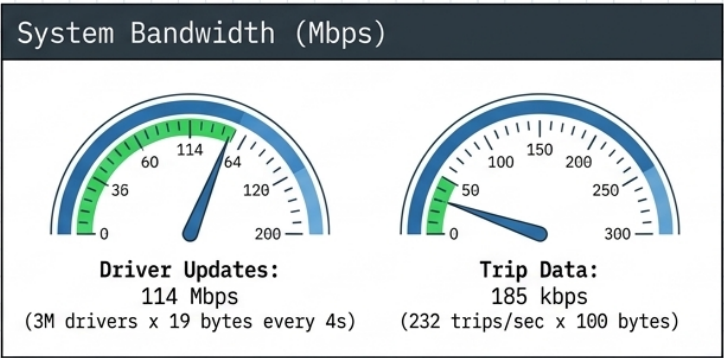
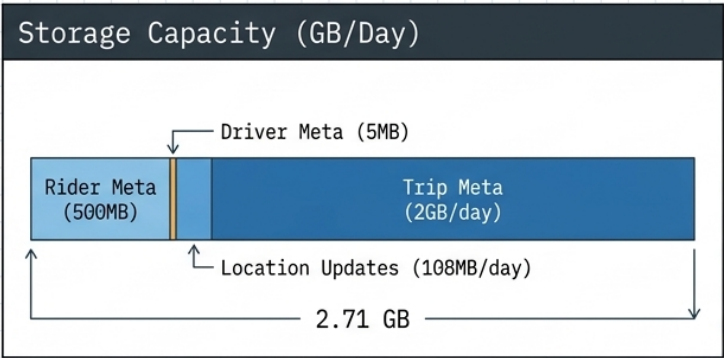
Daily Trips
20 Million

Telemetry Frequency
1 Location Update per
Active Driver / **4** Seconds

Architectural Friction: Balancing Requirements & Constraints



Telemetry Dashboard: Resource Estimations



Rider API Contracts

findNearbyDrivers

Payload: riderID, lat, long

Function: Identifies available drivers within a dynamic radius.

requestRide

Payload: riderID, lat, long, dropOfflat, dropOfflong, typeOfVehicle

Function: Initiates request and preferred vehicle class.

showETA

Payload: rider_lat, rider_lng, destination_lat, destination_lng, ride_type

Function: Calculates ETA to pickup point.

Driver & Trip Execution APIs

updateDriverLocation

Payload: driverID, oldlat, oldlong, newlat, newlong

Why oldlat / oldlong?

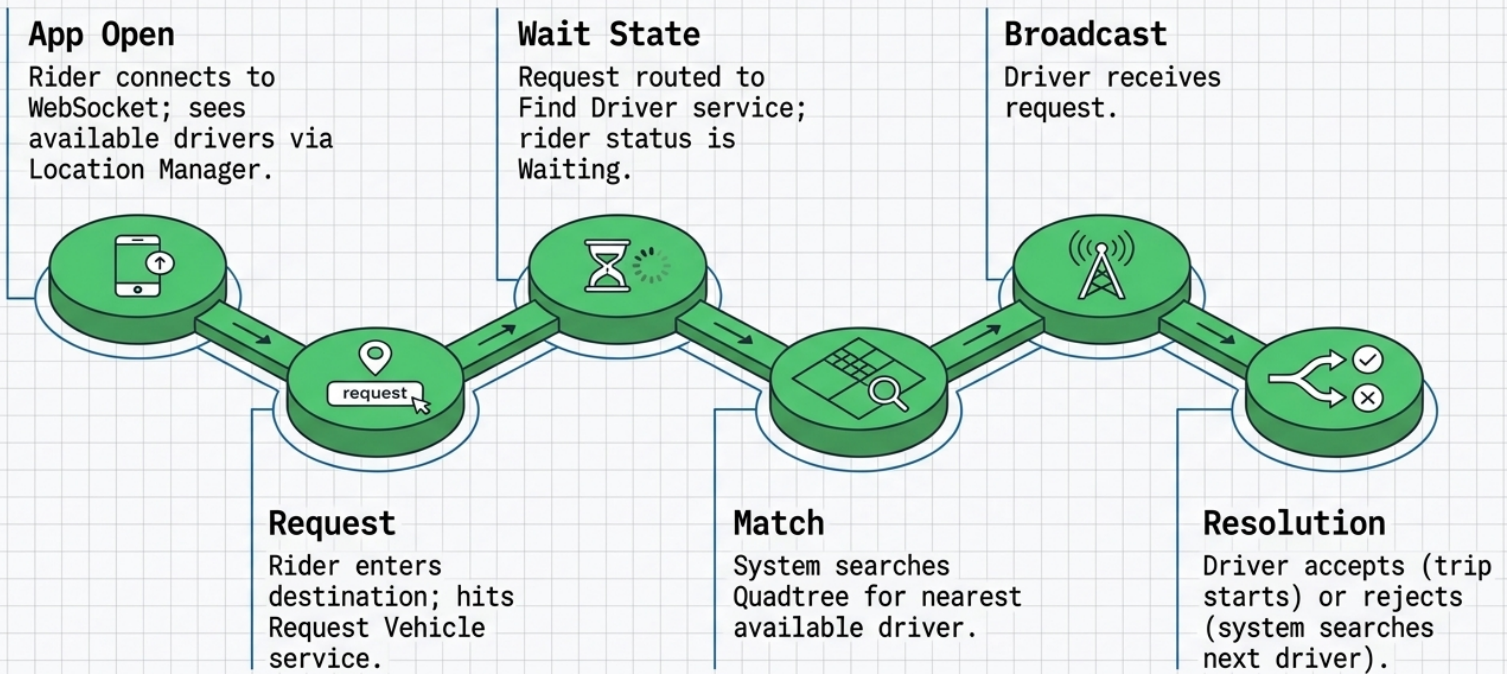
Enables the service to reconstruct the route if network issues cause local queuing of data packets. Low-bandwidth clients can omit these.

confirmPickup & showTripUpdates

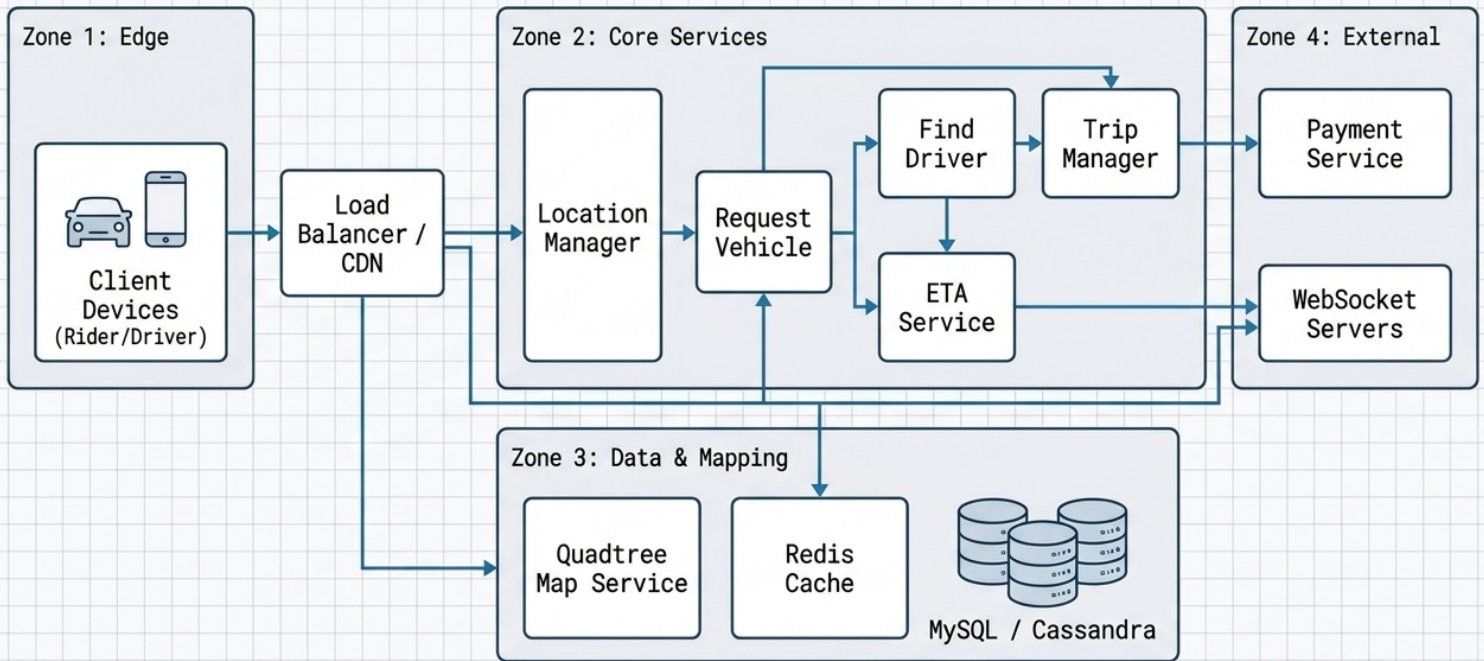
Payload: tripID, driverlat, driverlong, time_elapsed

Function: Signals trip start and streams real-time status to both parties.

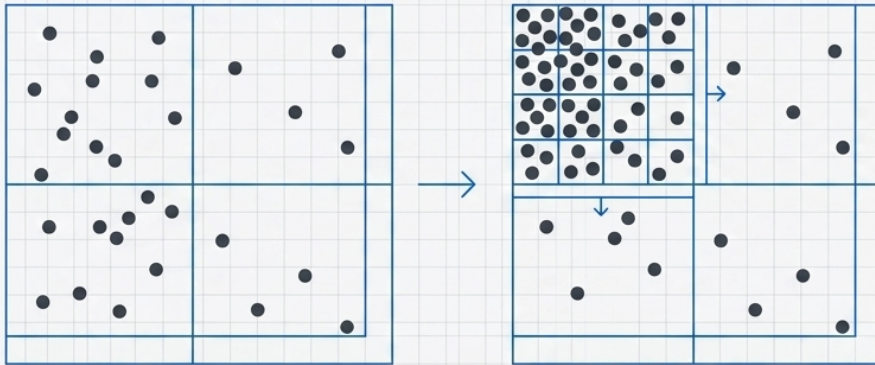
Life of a Request: Sequential Flow



High-Level System Architecture



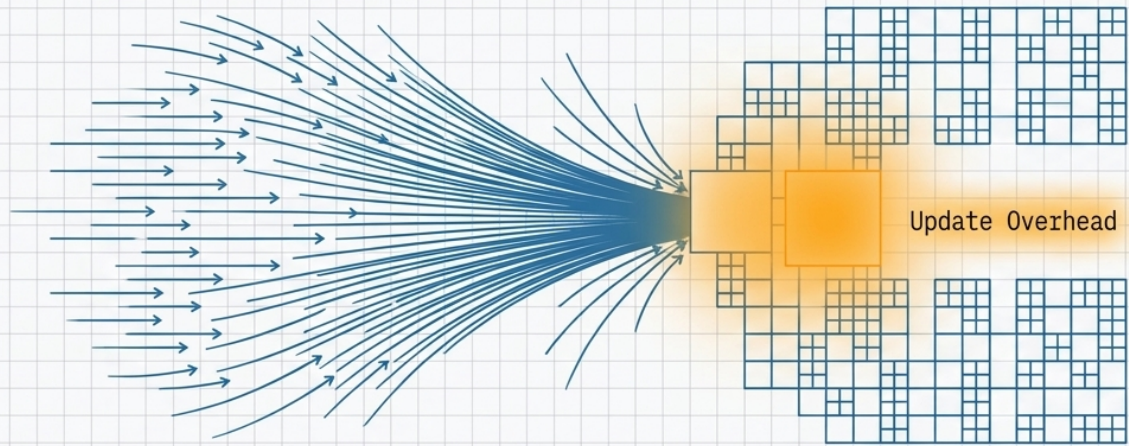
Spatial Partitioning via Quadrees



Concept: The map is partitioned into hierarchical spatial regions. When driver density in a region exceeds a threshold, the leaf node subdivides into four child regions.

Function: Redistributes drivers to maintain fast search times for the `findNearbyDrivers` service.

The Dynamic Update Bottleneck



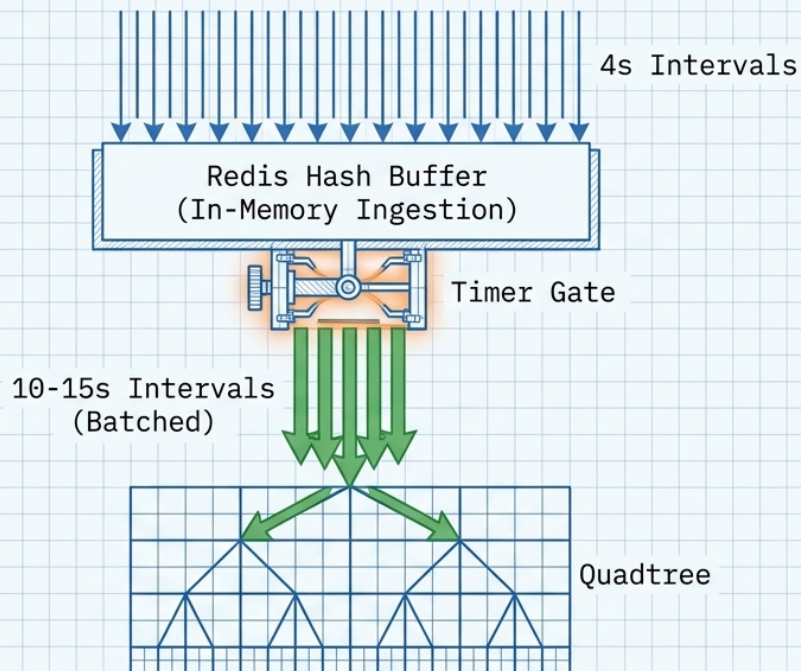
The Problem

Standard quadtrees were not designed for 3 million active drivers updating every 4 seconds.

The Cost

If a driver crosses region boundaries, the system must delete the old entry, insert the new one, and potentially subdivide regions. Doing this every 4 seconds per driver creates catastrophic update overhead.

Absorbing Load with Redis Batching

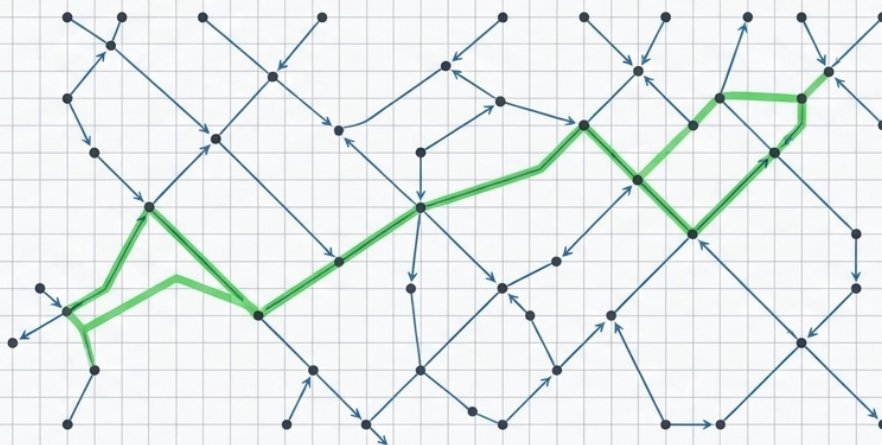


The Solution: Decouple the telemetry ingestion from the spatial indexing.

Mechanism: Updates write to a Redis hash every 4 seconds (fast, low-latency).

A separate process flushes these updates to the Quadtree in batches every 10-15 seconds, drastically reducing tree structural churn.

The ETA Calculation Engine



Component 1

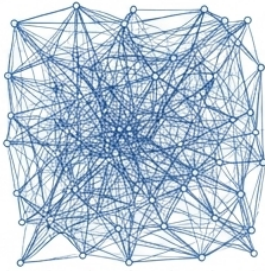
Calculate the shortest route on a road network (Graph representation where nodes = intersections, edges = road segments with one-way rules and turn limits).

Component 2

Compute the time required to travel the route (Applying real-time traffic data as edge weights).

Routing at Scale: Beyond Dijkstra

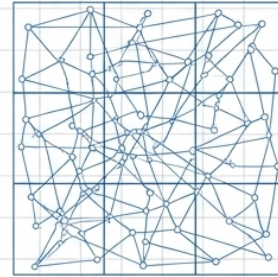
Standard Dijkstra



Mechanism: Operates on an unprocessed graph.

Verdict: Impractical at Uber scale. Too slow to compute massive graphs on the fly.

Partitioned Contraction Hierarchies

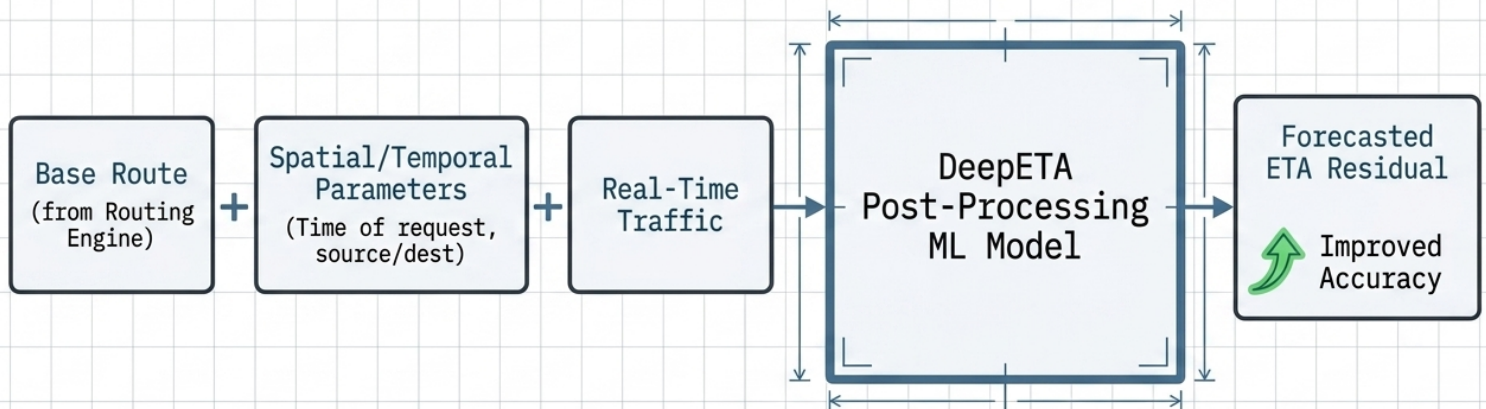


Mechanism: The graph is preprocessed and split into smaller, independent cells.

Advantage: Computes candidate routes in parallel across boundaries.

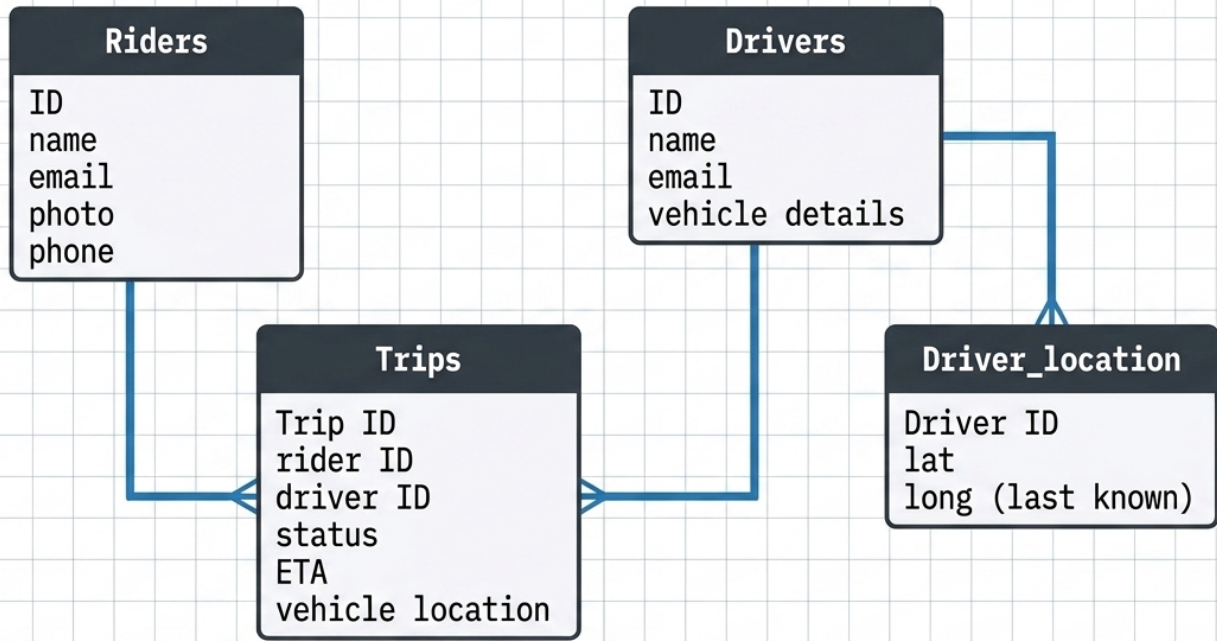
Result: Drastically reduces query latency; overall response time drops to ~1 second assuming balanced workloads.

DeepETA: Post-Processing Residuals



Concept: The routing engine provides the base path, but DeepETA leverages historical and real-time contextual data to predict the final ETA residual, significantly improving production metrics.

Core Data Storage Schema



Polyglot Persistence: Why not one database?

Apache Cassandra (NoSQL)

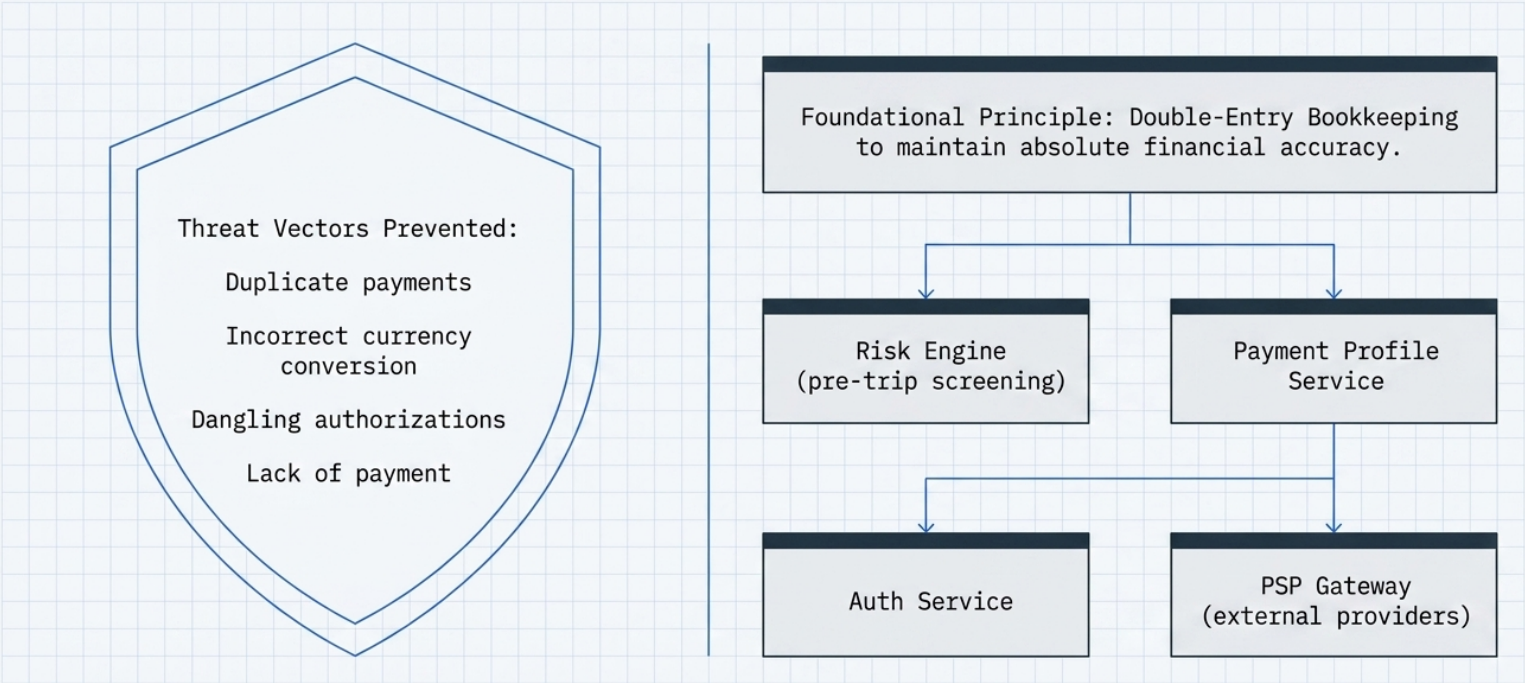
- **Use Case:** Driver's last known location, completed trip records.
- **Why:** High write volume, data grows infinitely, requires massive horizontal scalability and high availability.

MySQL (Relational)

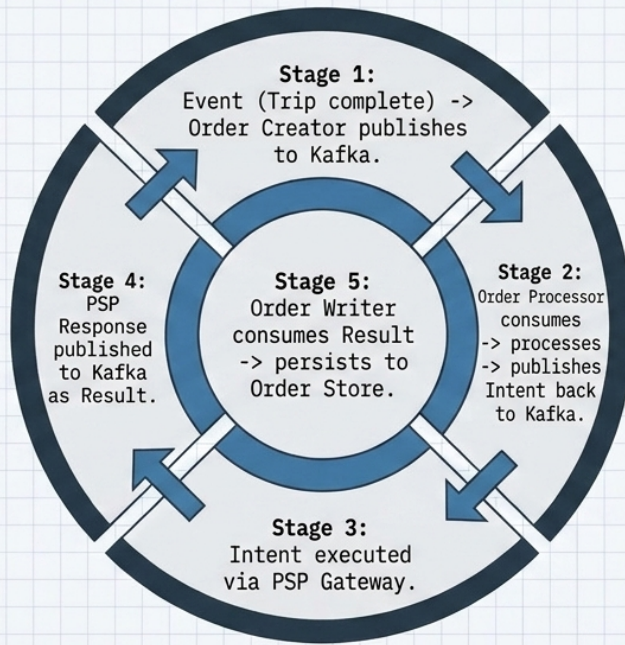
- **Use Case:** In-progress active trips.
- **Why:** Requires strong consistency, supports frequent status updates, trip data is highly relational during the active lifecycle.

Note: Uber has recently migrated toward Google Cloud Spanner to achieve global transactions with strongly consistent reads.

Payment Infrastructure & Risk



Asynchronous Payment Processing via Kafka



Why Kafka?

- Provides pub-sub queuing, fault-tolerant record storage, and decouples internal systems from slow external bank APIs.

The Anatomy of Platform Fraud

The Challenge: Fraud methods evolve like zero-day exploits. Minor percentages of fraud cause disproportionate revenue loss.

threat profile



Deliberately inefficient routing to inflate time/distance.

threat profile



GPS spoofing or fake location applications.

threat profile



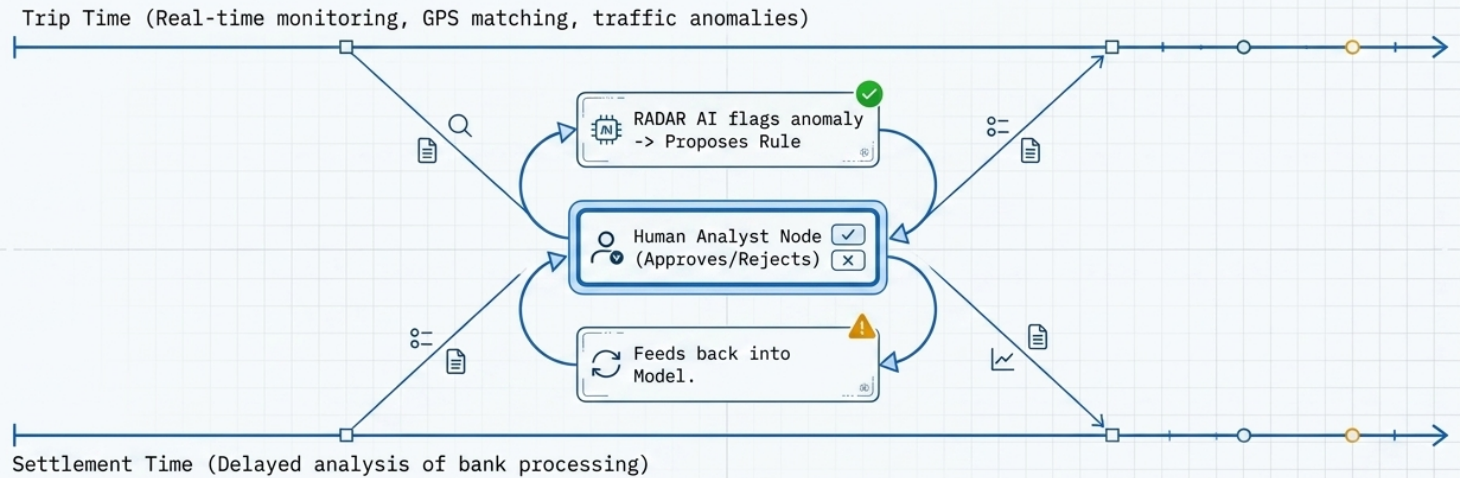
Forced rider cancellations (accepting with no intent to drive).

threat profile



Unwarranted fee claims (e.g., fake cleaning fees).

RADAR: Human-Assisted AI Detection



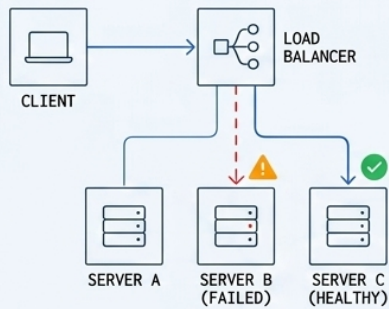
Why Human-in-the-Loop?

While it limits horizontal scalability, human oversight is required for accountability, bias mitigation, and reducing false positives on automated bans.

Fault Tolerance & Resilience Playbook

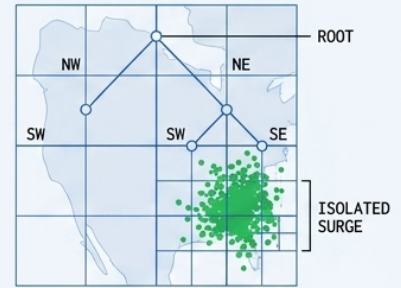
Network Drops

Persistent WebSocket connections via Load Balancers. If a server fails, the client seamlessly reconnects to a healthy instance.



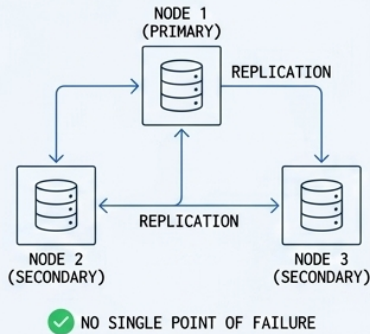
Data Partitioning

Quadtrees reduce the search space globally, isolating geographic surges.



Database Resilience

Cassandra primary-secondary replication removes single points of failure.



Offline Survival

If both rider and driver lose connectivity, the trip continues using local phone storage to cache route data until reconnection.

